

Exercício Programa 2

MAC0422 - Sistemas Operacionais (2026)

Simulador de Corrida Madison usando Barreira de Sincronização



Apresentação de Slides

Roteiro

3 - Decisões de Controle de Acesso ao Velódromo

4 - Como foi feita a Automação dos Testes

5 a 10 - Gráficos Comparativos

11 - Interpretação dos Resultados

12 - Especificações da Máquina usada e Conclusões

13 - Referências

Controle de Acesso ao Velódromo

- ❑ Para controlar o acesso ao velódromo na abordagem eficiente, um ciclista deve **ganhar o mutex de duas colunas**: da sua *coluna atual* (cujo número se refere ao “metro” em que está), e então o da *coluna seguinte*. A cada ciclo de 60 milissegundos a ordem de movimento é respeitada:
 - 1 - Tenta avançar mantendo-se na mesma pista
 - 2 - Tenta ultrapassar se estiver rápido o suficiente, e houver oportunidade por pista externa
 - 3 - Tenta descer para uma pista mais interna
- ❑ Os dois mutexes são necessários para que ninguém atrás tente sobrescrever sua posição atual, nem que o mesmo seja feito com quem está na frente. São liberados após as tentativas.
- ❑ Teoricamente mais eficiente que **um único mutex** para todo o velódromo por reduzir esperas desnecessárias: uma ciclista no meio do velódromo não tenta escrever e competir por mutex com uma ciclista nas extremidades.
- ❑ Teoricamente mais eficiente que **um mutex para cada posição** do velódromo por ser mais simples de implementar e reduzir probabilidade de deadlocks - seriam mais mutexes para coordenar, e ainda seria necessário ganhar dois ou mais mutexes por thread a cada ciclo.

Automação dos Testes

```
#!/bin/bash
(...)
```

```
nice -n 15 /usr/bin/time -f "%C,%U,%S,%P,%M,%R,%w,%c,%x" timeout 600s
./ep2 10 150 10 i 2>>$LOG
nice -n 15 /usr/bin/time -f "%C,%U,%S,%P,%M,%R,%w,%c,%x" timeout 600s
./ep2 10 150 40 i 2>>$LOG
nice -n 15 /usr/bin/time -f "%C,%U,%S,%P,%M,%R,%w,%c,%x" timeout 600s
./ep2 10 150 70 i 2>>$LOG
```

```
nice -n 15 /usr/bin/time -f "%C,%U,%S,%P,%M,%R,%w,%c,%x" timeout 600s
./ep2 10 600 10 i 2>>$LOG
nice -n 15 /usr/bin/time -f "%C,%U,%S,%P,%M,%R,%w,%c,%x" timeout 600s
./ep2 10 600 145 i 2>>$LOG
nice -n 15 /usr/bin/time -f "%C,%U,%S,%P,%M,%R,%w,%c,%x" timeout 600s
./ep2 10 600 280 i 2>>$LOG
```

```
nice -n 15 /usr/bin/time -f "%C,%U,%S,%P,%M,%R,%w,%c,%x" timeout 600s
./ep2 10 1350 10 i 2>>$LOG
nice -n 15 /usr/bin/time -f "%C,%U,%S,%P,%M,%R,%w,%c,%x" timeout 600s
./ep2 10 1350 255 i 2>>$LOG
nice -n 15 /usr/bin/time -f "%C,%U,%S,%P,%M,%R,%w,%c,%x" timeout 600s
./ep2 10 1350 500 i 2>>$LOG
```

```
(...)
```



Esse conjunto exemplo de 9 testes foi repetido 30 vezes, mais 30 para a abordagem eficiente, totalizando 540 testes, levando aprox. 2 horas para realizá-los. Os resultados foram anexados a um log.



Apenas o terminal com a execução do EP esteve aberto, para reproduzir as condições de simulação mais próximas possíveis; ‘time’ monitorou as métricas desejadas, enquanto ‘timeout’ garantia que nenhum teste congelaria.



Todos com o mesmo número de voltas ($n = 10$). O velódromo pequeno tinha $d = 150\text{m}$, o médio tinha $d = 600\text{m}$ (4x maior que o pequeno), e o grande tinha $d = 1350\text{m}$ (9x maior que o pequeno).



Contagem pequena de ciclistas $k = 10$ qualquer que fosse o tamanho do velódromo, e como havia limitação de k a depender de d , definiu-se para cada metragem “muitos” ciclistas como algo “um pouco menor que a metade de d ”: $k = 70$ se $d = 150$, $k = 280$ se $d = 600$, $k = 500$ se $d = 1350$.



Nos casos de quantidade “normal” de ciclistas, foi usado a média aritmética entre “poucos” e “muitos” para cada metragem.

Tempo (s) - Velódromo Pequeno

INGÊNUA

$$\bar{x}_t(150, 10) = 0.61$$

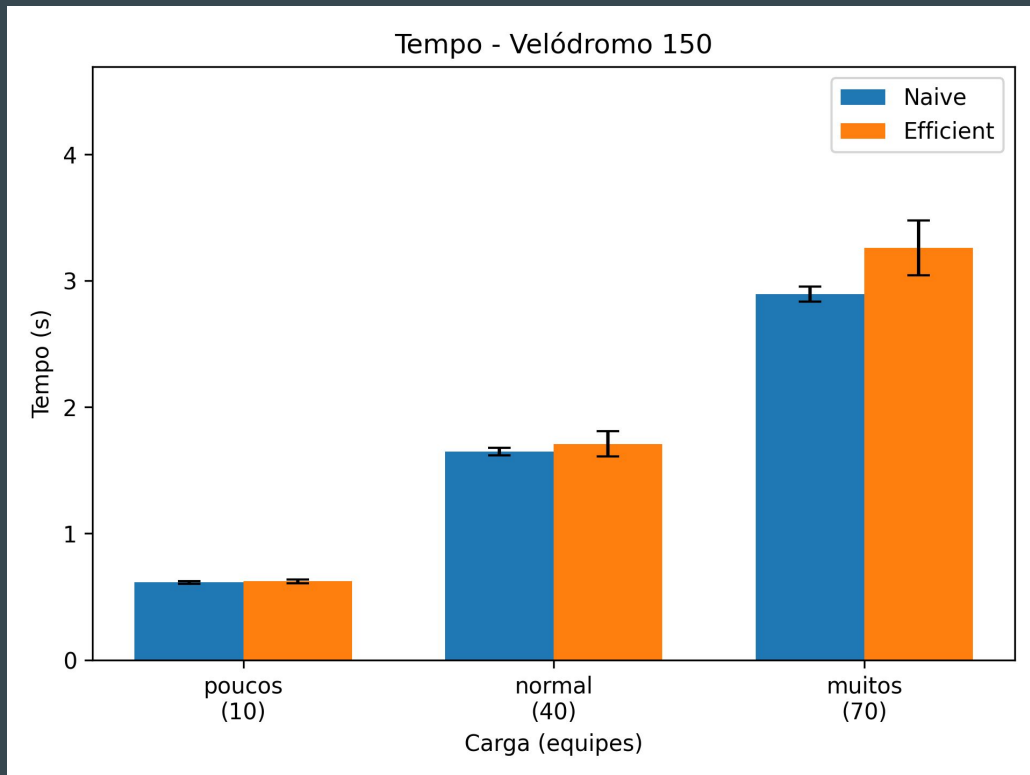
$$s_t(150, 10) = 0.03$$

$$\bar{x}_t(150, 40) = 1.65$$

$$s_t(150, 40) = 0.08$$

$$\bar{x}_t(150, 70) = 2.89$$

$$s_t(150, 70) = 0.16$$



EFICIENTE

$$\bar{x}_t(150, 10) = 0.62$$

$$s_t(150, 10) = 0.04$$

$$\bar{x}_t(150, 40) = 1.71$$

$$s_t(150, 40) = 0.26$$

$$\bar{x}_t(150, 70) = 3.26$$

$$s_t(150, 70) = 0.58$$

*Obs.: o tempo mostrado aqui é o tempo total, ou seja, usr time + sys time.

Tempo (s) - Velódromo Médio

INGÊNUA

$$\bar{x}_t(600, 10) = 1.75$$

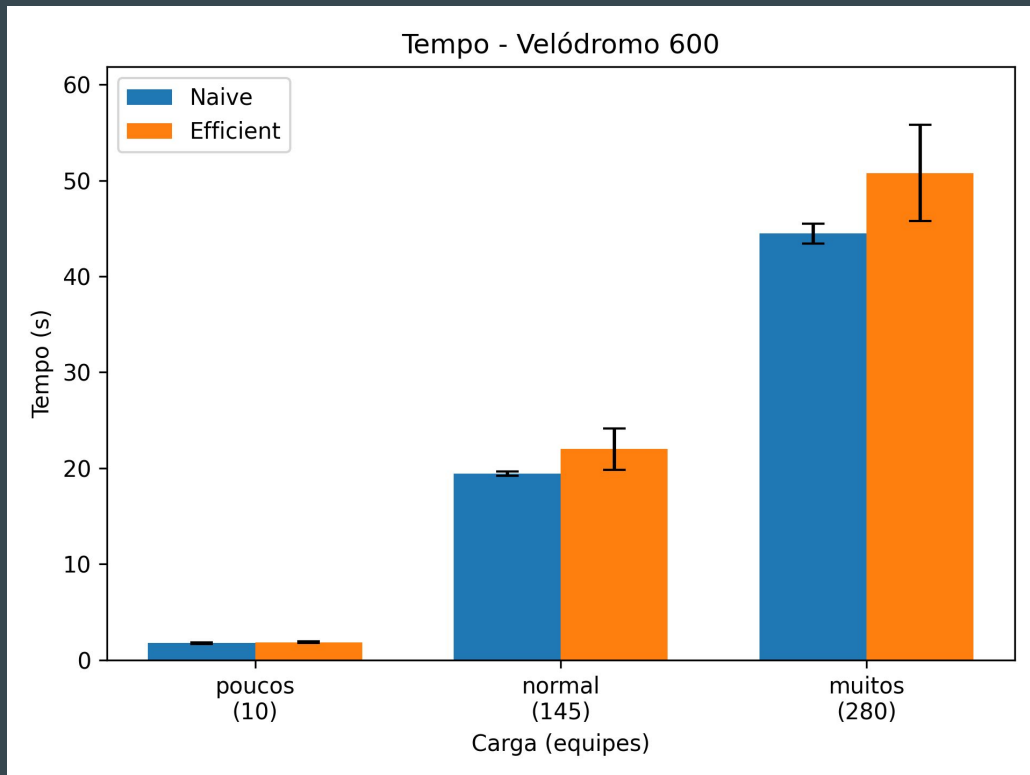
$$s_t(600, 10) = 0.16$$

$$\bar{x}_t(600, 145) = 19.44$$

$$s_t(600, 145) = 0.55$$

$$\bar{x}_t(600, 280) = 44.47$$

$$s_t(600, 280) = 2.76$$



EFICIENTE

$$\bar{x}_t(600, 10) = 1.85$$

$$s_t(600, 10) = 0.20$$

$$\bar{x}_t(600, 145) = 21.98$$

$$s_t(600, 145) = 5.76$$

$$\bar{x}_t(600, 280) = 50.79$$

$$s_t(600, 280) = 13.45$$

*Obs.: o tempo mostrado aqui é o tempo total, ou seja, usr time + sys time.

Tempo (s) - Velódromo Grande

INGÊNUA

$$\bar{x}_t(1350, 10) = 3.22$$

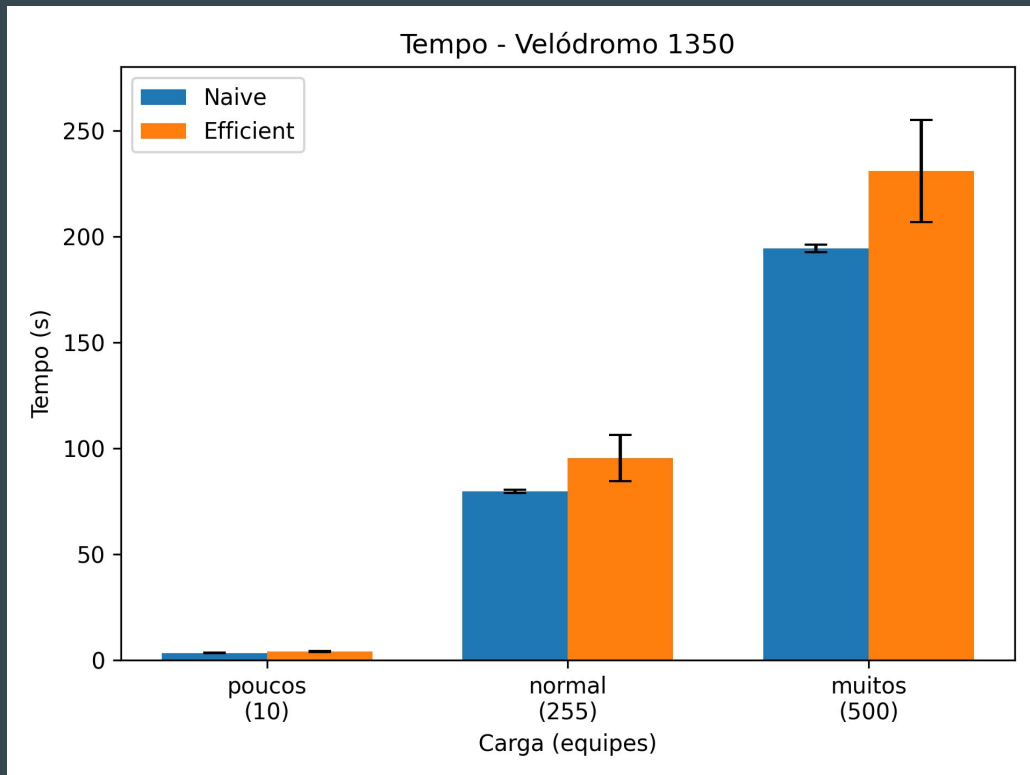
$$s_t(1350, 10) = 0.40$$

$$\bar{x}_t(1350, 255) = 79.66$$

$$s_t(1350, 255) = 2.33$$

$$\bar{x}_t(1350, 500) = 194.49$$

$$s_t(1350, 500) = 4.81$$



EFICIENTE

$$\bar{x}_t(1350, 10) = 3.88$$

$$s_t(1350, 10) = 0.66$$

$$\bar{x}_t(1350, 255) = 95.38$$

$$s_t(1350, 255) = 29.33$$

$$\bar{x}_t(1350, 500) = 231.01$$

$$s_t(1350, 500) = 64.53$$

*Obs.: o tempo mostrado aqui é o tempo total, ou seja, usr time + sys time.

Memória (kB) - Velódromo Pequeno

INGÊNUA

$$\bar{x}_{\text{mem}}(150, 10) = 2156$$

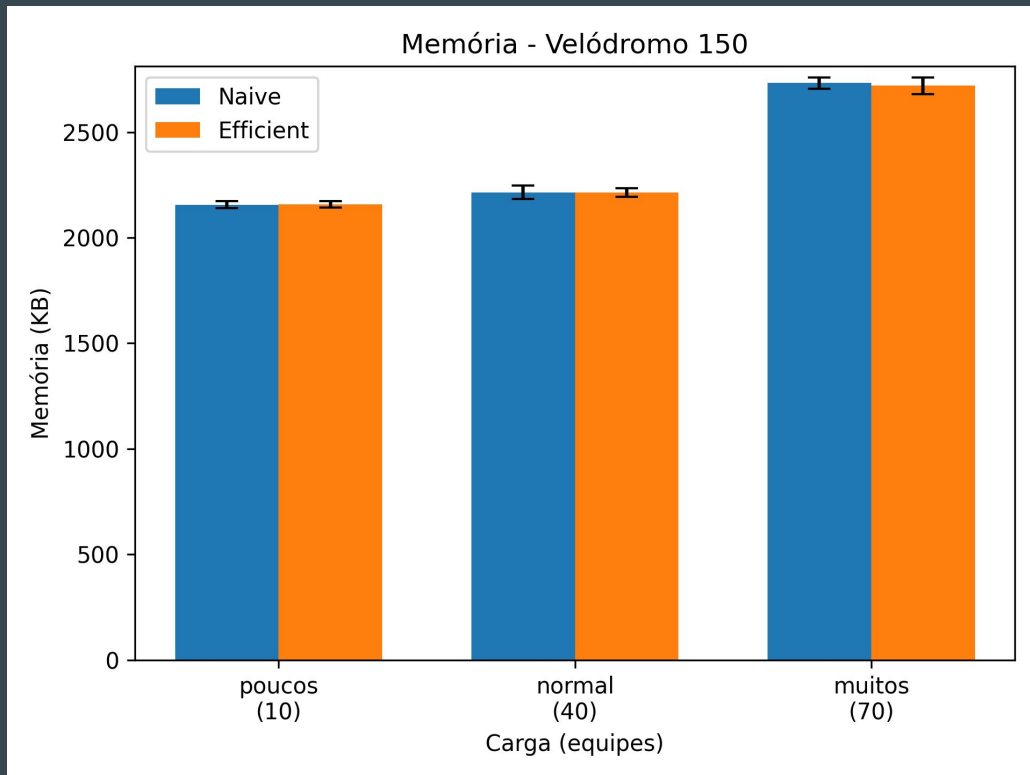
$$s_{\text{mem}}(150, 10) = 45$$

$$\bar{x}_{\text{mem}}(150, 40) = 2214$$

$$s_{\text{mem}}(150, 40) = 85$$

$$\bar{x}_{\text{mem}}(150, 70) = 2732$$

$$s_{\text{mem}}(150, 70) = 70$$



EFICIENTE

$$\bar{x}_{\text{mem}}(150, 10) = 2159$$

$$s_{\text{mem}}(150, 10) = 42$$

$$\bar{x}_{\text{mem}}(150, 40) = 2214$$

$$s_{\text{mem}}(150, 40) = 57$$

$$\bar{x}_{\text{mem}}(150, 70) = 2720$$

$$s_{\text{mem}}(150, 70) = 106$$

Memória (kB) - Velódromo Médio

INGÊNUA

$$\bar{x}_{\text{mem}}(600, 10) = 2146$$

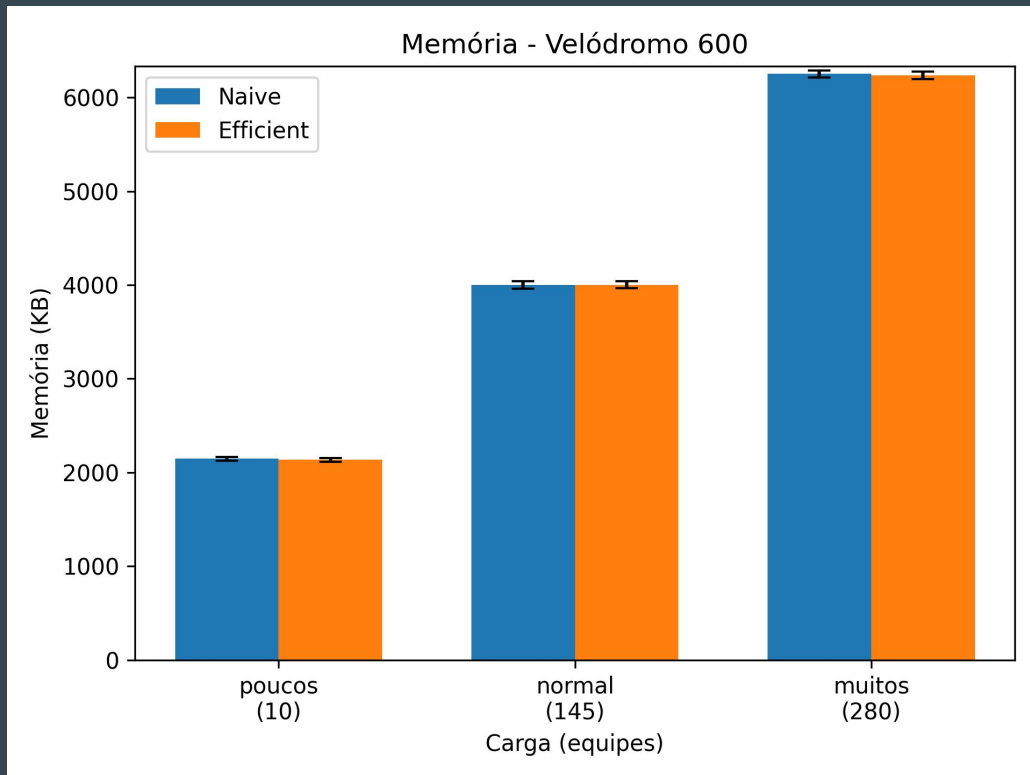
$$s_{\text{mem}}(600, 10) = 59$$

$$\bar{x}_{\text{mem}}(600, 145) = 3999$$

$$s_{\text{mem}}(600, 145) = 108$$

$$\bar{x}_{\text{mem}}(600, 280) = 6252$$

$$s_{\text{mem}}(600, 280) = 100$$



EFICIENTE

$$\bar{x}_{\text{mem}}(600, 10) = 2134$$

$$s_{\text{mem}}(600, 10) = 59$$

$$\bar{x}_{\text{mem}}(600, 145) = 4002$$

$$s_{\text{mem}}(600, 145) = 98$$

$$\bar{x}_{\text{mem}}(600, 280) = 6236$$

$$s_{\text{mem}}(600, 280) = 105$$

Memória (kB) - Velódromo Grande

INGÊNUA

$$\bar{x}_{\text{mem}}(1350, 10) = 2141$$

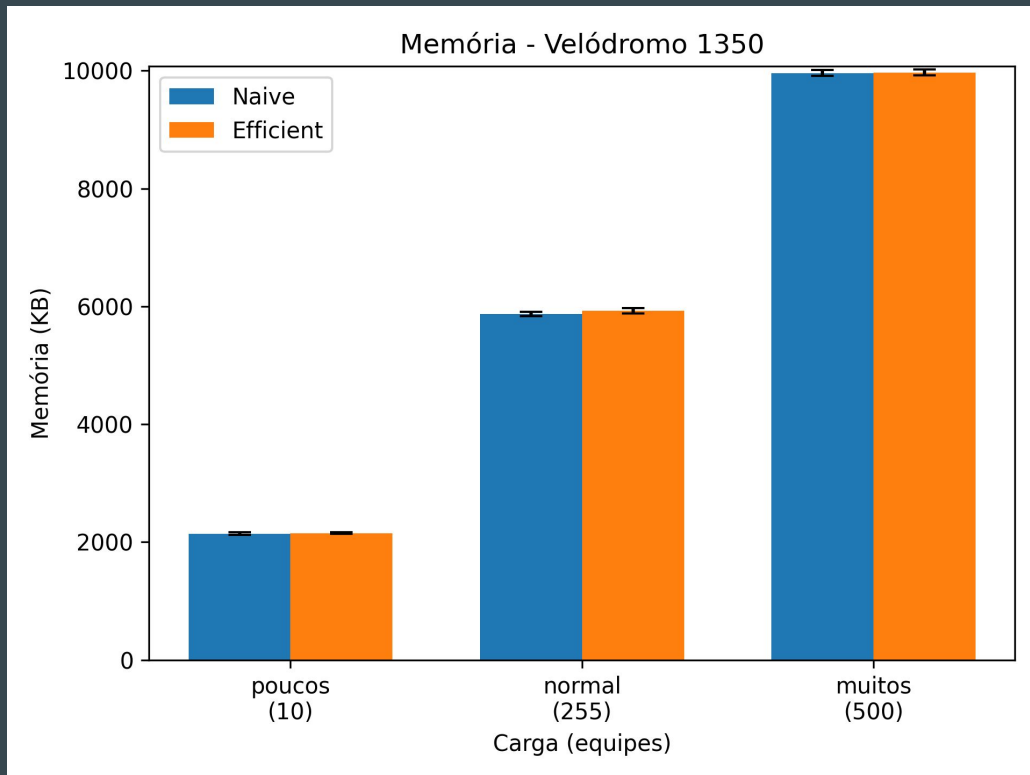
$$s_{\text{mem}}(1350, 10) = 61$$

$$\bar{x}_{\text{mem}}(1350, 255) = 5871$$

$$s_{\text{mem}}(1350, 255) = 103$$

$$\bar{x}_{\text{mem}}(1350, 500) = 9962$$

$$s_{\text{mem}}(1350, 500) = 139$$



EFICIENTE

$$\bar{x}_{\text{mem}}(1350, 10) = 2153$$

$$s_{\text{mem}}(1350, 10) = 45$$

$$\bar{x}_{\text{mem}}(1350, 255) = 5925$$

$$s_{\text{mem}}(1350, 255) = 118$$

$$\bar{x}_{\text{mem}}(1350, 500) = 9972$$

$$s_{\text{mem}}(1350, 500) = 128$$

Entradas: pequenas, médias e grandes + Resultados

- ❑ Todas métricas dentro do intervalo de confiança (?) Fato é que na abordagem eficiente com quantidade normal ou alta de ciclistas, os dados foram mais ruidosos...
- ❑ Quanto mais ciclistas, menos ruidosos foram os dados de memória usada, com tendência de queda no desvio padrão relativo, mas o mesmo não ocorreu com o tempo.
- ❑ Os dados sugerem aumento do tempo como algo entre linear e quadrático com o aumento de ciclistas, além de aumento linear em memória, considerando que os acréscimos em k são iguais em cada tamanho de pista (escala linear).
- ❑ Os dados sugerem aumento do tempo com o aumento do velódromo, mas não de forma expressiva como com o incremento do número de equipes.
- ❑ As duas abordagens tiveram números próximos e com margens de erro sobrepostas para todas as métricas!
- ❑ Outras métricas úteis capturadas com `/usr/bin/time`:

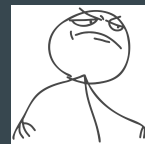
Page faults: esteve na ordem de grandeza de 10^2 quando tínhamos algo entre 10 e 100 equipes, e ordem de grandeza 10^3 quando tínhamos algo entre 100 e 500 equipes. ☁ *Reforça que variáveis associadas ao uso de memória só foram impactadas de forma expressiva por k .*

Context Switches (voluntários): esteve na ordem de grandeza de 10^5 a 10^6 em velódromo pequeno, e por volta de 10^7 nos outros tamanhos. Parece ter sido afetado tanto pelo tamanho do velódromo, quanto pelo número de equipes, e não pela abordagem. ☁ *Possível razão: threads ciclistas bloqueiam mais vezes na barreira e nos mutexes tendo mais metros para percorrer, aumentando essa contagem.*

Configurações Utilizadas e Conclusões



Intel Core i5-13420H (12 cores) 13ª gen,
Fedora Linux 43 (Workstation Edition),
(12 núcleos lógicos, 8 físicos)



Impacto do número de ciclistas

- Desvantagem de ter sido calculado de forma relativa aos tamanhos de velódromo, logo as comparações necessitam de contextualização cuidadosa.
- Mesmo assim, bem relevante, uma vez que esclareceu o impacto do número de threads numa execução de um processo, principalmente no tempo. Isso era esperado.
- Aumento no uso de memória também previsível, já que é custoso armazenar todos os dados dos ciclistas, além do gerenciamento de memória pelo SO ser mais complexo quando se envolve mais threads.

Impacto do tamanho do velódromo

- Possível perceber impacto no tempo por dois argumentos:
1 - a média de tempo aumentou com o acréscimo de d ao fixar $k = 10$;
2 - a comparação $\bar{x}_t(1350, 255) = 79.66 > \bar{x}_t(600, 280) = 44.47$.
Mesmo com mais ciclistas, esta última foi mais rápida por ter o velódromo 2,25x menor!
- Sem impacto significativo em termos de memória, tanto que ao aumentar d e fixar $k = 10$, sempre se manteve na casa dos 2 kB. Esperado, já que a sua representação usa apenas ponteiros para os ciclistas - 8 bytes cada.

“Impacto” da abordagem

- Fora do esperado, já que mesmo com um mutex global a abordagem ingênua não foi inferior em tempo, e em memória o consumo foi similar. Provavelmente devido a custo de sincronização de threads pelo SO, barreiras e contenções terem dominado as métricas.
- Dois mutexes por thread? Pode ser que os overheads de espera ao ganhar um mutex e **perder** outro gera acúmulos de maneira constante, principalmente com um velódromo denso - k próximo de $\lfloor d/2 \rfloor - 1$ - onde dificilmente ciclistas conseguem correr a 60 km/h.

Referências

Livros:

Andrew S. Tanenbaum, "Sistemas operacionais modernos", Pearson Education do Brasil Pearson, 2010.

Chatbots (sem código gerado, apenas discussão):

<https://claude.ai/share/eaec6503-2f00-4bfb-b633-ba3633e7e2df>

Manpages e Docs:

<https://man7.org/linux/man-pages/man1/time.1.html>

<https://man7.org/linux/man-pages/man2/nanosleep.2.html>

<https://en.cppreference.com/w/c/header/stdatomic.html>

<https://docs.oracle.com/cd/E19253-01/816-5137/gfwek/index.html>

Métricas:

<https://courses.cs.duke.edu/cps110/spring01/slides/interleave/tsld008.htm>

<https://stackoverflow.com/questions/2234537/what-causes-page-fault-and-how-to-minimize-them>

https://en.wikipedia.org/wiki/Thundering_herd_problem

Evitando Deadlocks:

<https://www.codequoi.com/en/threads-mutexes-and-concurrent-programming-in-c/#the-danger-of-data-races>

<https://stackoverflow.com/questions/874432/when-is-it-safe-to-destroy-a-pthread-barrier>